HERMES

From Chatbots to Long-Running AI Agents

How Persistent Memory and Skills Change AI Workflows

Alexandra Arguello Saenz · Enterprise Solutions Architect

github.com/alexarguello/arturito

The question everyone's asking…

"I already use ChatGPT. Why do I need an agent?"

Forgets everything
after the chat ends

Can't run tasks
while you sleep

No real autonomy.
Just a smarter search box.

THE CORE DIFFERENCE

Chatbot vs Agent

Chatbot

- Stateless — forgets after each session
- Reactive — waits for your prompt
- Single model call per response
- Lives inside one window only
- You manage all the context

Hermes Agent

- **Persistent memory across sessions**
- **Proactive — runs scheduled tasks**
- **Multi-step reasoning + sub-agents**
- **Lives on your server, all channels**
- **Grows its own skill library over time**

THE PROBLEM

Why the Chatbot Model Breaks Down

Real engineering work doesn't fit in a single chat window.

Fragile Session Memory

Short-term memory that resets — costly and unreliable across extended work sessions.

Repetitive Context Sharing

Explain your project from scratch every session. Tedious and error-prone.

Reactive Interaction Model

Responds only when prompted. Can't monitor, follow up, or run unattended.

No Learning from Outcomes

Each interaction is isolated. Chatbots don't adapt or improve from your feedback.

WHAT IS HERMES

Not a chatbot wrapper.

A persistent autonomous agent.

Open Source · MIT License · v0.12.0 · Nous Research

Persistent Memory

Never forgets how it solved your problems

Auto-Generated Skills

Builds reusable skills automatically over time

Scheduled Automations

Natural language cron — runs unattended 24/7

Delegates & Parallelizes

Isolated sub-agents with zero context bleed

Real Sandboxing

Docker, SSH, Modal, Singularity backends

Full Web Control

Search, browser automation, vision, TTS

MENTAL MODEL

Think of it as a junior engineer that never forgets.

Junior Engineer

- Follows your instructions
- Handles repetitive tasks
- Improves through repetition
- Works best with defined scope
- Remembers what worked last time

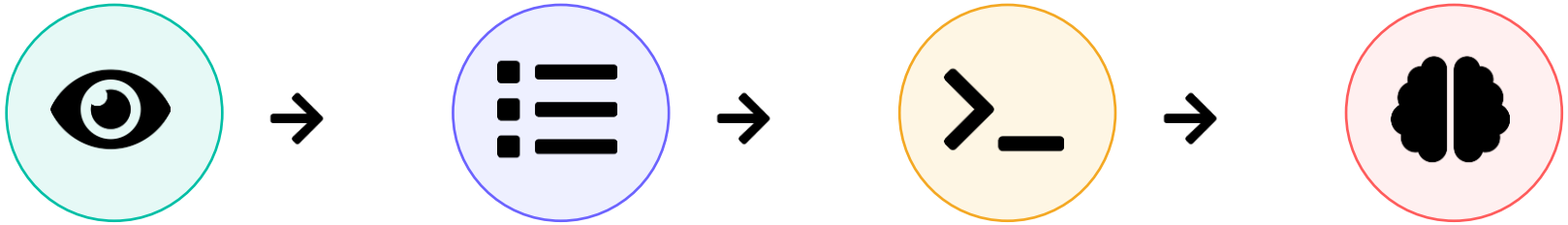
Hermes Agent

- Follows your instructions
- Handles repetitive tasks
- Improves through feedback loops
- Works best with clear goals
- Remembers EVERYTHING it has done

ARCHITECTURE

The Agent Loop

Observe → Plan → Act → Learn — repeating forever



OBSERVE

Gathers input from users, system state, and past memories

PLAN

Breaks goals into steps, selects tools or sub-agents

ACT

Interacts with filesystems, terminals, browsers, APIs

LEARN

Records outcomes to improve every future cycle

↩ The loop runs continuously — every completed cycle makes Hermes smarter and more efficient

PERSISTENCE

Memory as a First-Class Component

Why memory changes everything

- Stored on disk — survives restarts
- Facts, decisions, and project context preserved
- Selective recall reduces token cost
- System improves with every task completed
- No more re-explaining your stack every morning

Chatbot memory

Session 1 | Session 2 | Session 3

✕ Memory resets each time

Hermes memory

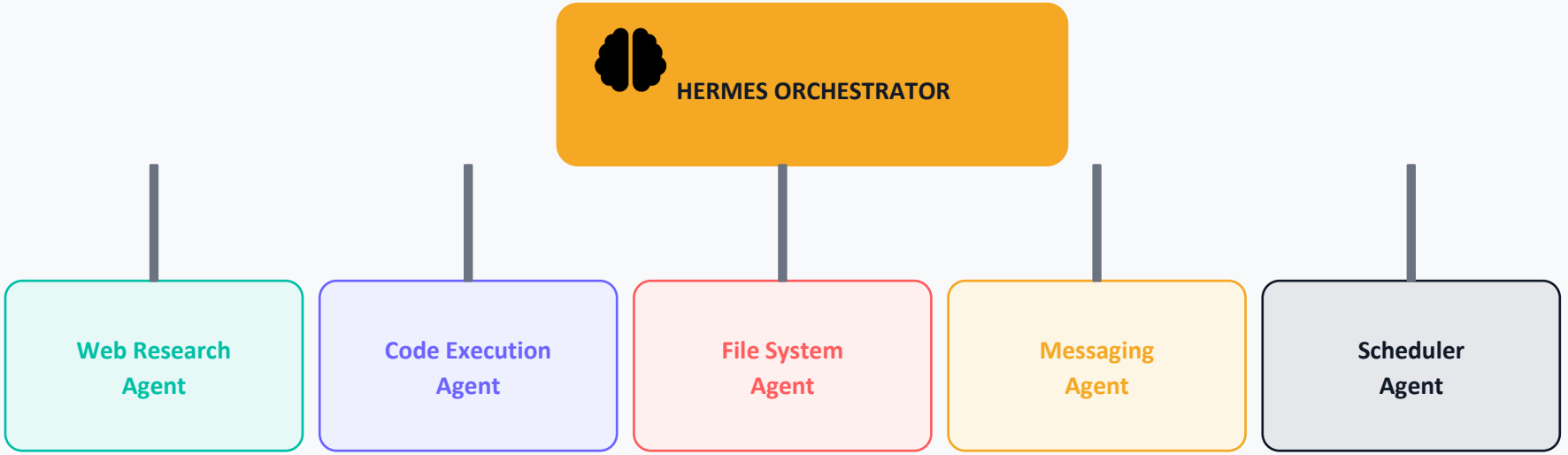
Day 1 | Day 7 | Day 30

✓ Grows smarter every session

ARCHITECTURE

Delegation Through Sub-Agents

Think microservices, but for AI tasks.



- Isolated context — no pollution
- Parallel execution
- Failures stay contained
- Scales like microservices

LIVE DEMO

Meet Arturito

My personal Hermes agent — named after R2-D2 in Spanish 🤖

```
$ curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash

$ hermes setup

# Arturito in action:

> "Send me a daily standup summary at 9am"

✓ Scheduled. Memory stored. I'll handle it.
```

Hermes v0.12	**Telegram + CLI**
Built on	Channel
Docker + VPS	**Daily driver**
Runs on	Purpose

github.com/alexarguello/arturito

USE CASES

What Hermes Does While You Sleep

Codebase Memory

Learns your repo structure, naming conventions, and past decisions. Picks up exactly where you left off — even weeks later.

Scheduled Reports

Daily standup summaries, weekly digests, deployment alerts — all triggered by natural language, no cron syntax needed.

Parallel Research

Spawns sub-agents to research multiple topics simultaneously. Synthesizes results and delivers a clean summary.

Anywhere Access

Message Arturito on Telegram, Slack, or Discord. Same agent, same memory, same skills — any platform.

THE BIGGER PICTURE

The Shift to Agentic Infrastructure

1960s–80s

Batch Processing

Run, wait, collect output

→

1990s–00s

Interactive Systems

Real-time input/output

→

2010s

API + Chatbots

Query → response pattern

→

NOW

Agentic AI

Persist, remember, improve

★ As models get smarter, the bottleneck shifts to integration, memory, and control — not the model itself. That's the bet Hermes is making.

OpenClaw Vs Hermes

OpenClaw – Connect & Execute

- Executes actions across systems
- Uses explicit / predefined skills
- Optimized for immediate results
- Best for: Quick setup, Cross-system workflows, Assistant-style interactions
- Broader interaction surface: Assistant coordinating with many external systems

Hermes – Learn & Improve

- Improves performance across tasks
- Generates and evolves skills
- Optimized for iterative improvement
- Best for: Repeated workflows, Long-term optimization, Adaptive agents
- Narrower interaction surface: Assistant refining how work gets done over time

Your Turn.

Build your own agent. Here's how to start.

1

Install Hermes

Locally (or on a VPS) and complete the guided setup.

2

Give it a real task

Ask Hermes to do one concrete thing via CLI or messaging (Telegram, Slack, etc.).
something you do daily

3

Watch it learn

Review memory saves and
auto-generated reusable skills

4

Make it always-on

Deploy to VPS, connect your messaging channels
Schedule one useful recurring task, then add tools or integrations as needed

5

Fork Arturito

Name it yours. Extend it.
Contribute back.

🌐 hermes-agent.nousresearch.com/docs — docs + quickstart
🌐 datacamp.com/tutorial/hermes-agent — Setup and Tutorial Guide

github.com/alexarguello/arturito -- Fork it. Make it yours.